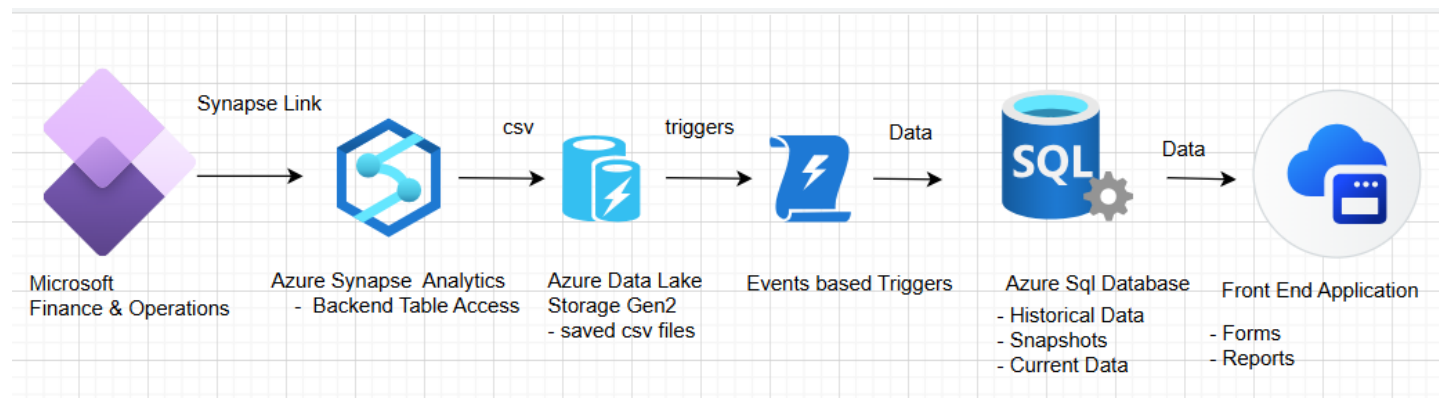


Extracting data from Microsoft Dynamics CRM

Stakeholders: Business Analysts, Business Development Team, leadership team members

Purpose: The main purpose of this project is to deliver the data from the Microsoft Finance & Operations in a near real time to avoid critical issues such as delayed financial visibility, inaccuracy in budget tracking and spending, cash flow management and any sort of operational inefficiencies.

High-level Architecture Diagram of the Technical Workflow



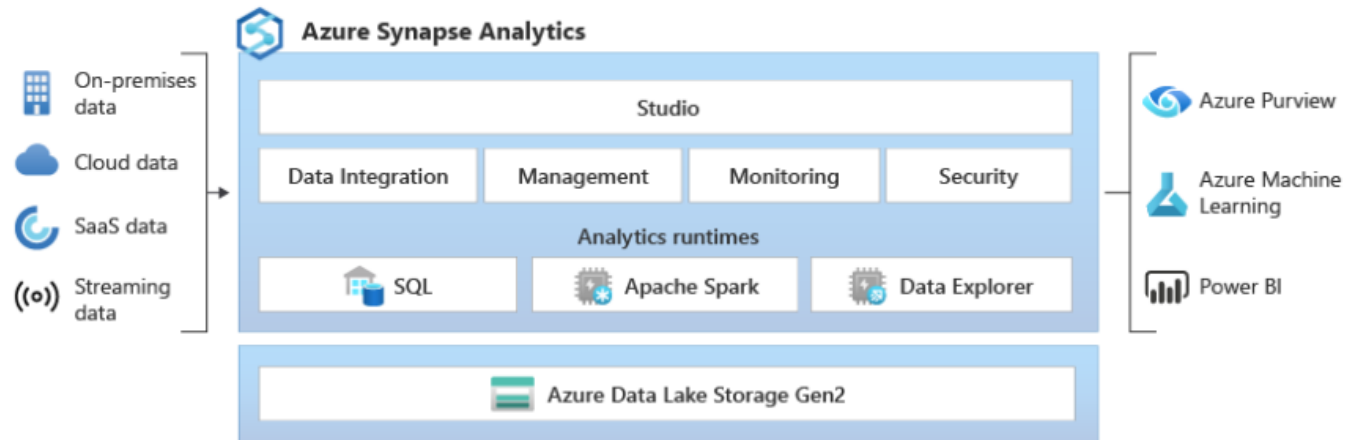
Technical Summary:

Cadence: This data architecture pipeline will run based on the events based triggers. Whenever any changes occurs in the Microsoft Finance & Operations tables, triggers pointed to those backend tables will run the trigger.

Microsoft Finance & Operations: Dynamics 365 Finance & Operations is a cloud-based enterprise resource planning software designed by Microsoft. This solution is designed as a collaborative platform for different valuable operations such as supply chain, financials, sales, purchase orders, inventory management. D365 F&O contains two major options to select from: Finance and Supply Chain Management. Some of the useful features of this solution are finance management, business processes management, planning, forecasting and inventory management, procurement and sourcing, order management and pricing, asset management.

Azure Synapse Analytics: Azure Synapse is an enterprise level analytics service provided by Microsoft that helps the users to accelerates time to data insight across data warehouses systems. Azure Synapse consist of SQL technologies used in enterprise data warehousing, Spark

technologies used for big data, Pipelines for data integration and ETL/ELT, and deep integration with other Azure services such as Power BI, CosmosDB, and AzureML.

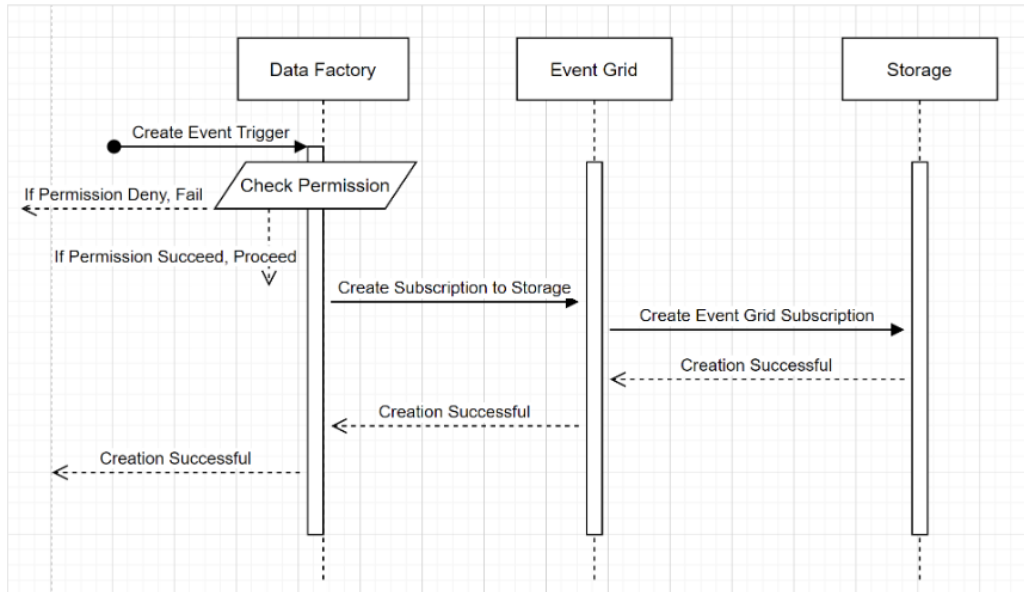


Azure Data Lake: To setup the Azure synapse link between the Microsoft Finance & Operations backend system Dataverse and Azure synapse Link, the official documentation from Microsoft has been used [Link](#) . The Azure Synapse Link for Dataverse is a service provided by microsoft to deliver scalable and high availability of data. Data is stored in the Azure Data Lake. It continuously exports the data and metadata to the Azure Data Lake. It supports initial and incremental writes of the data in the table and its metadata. Any sorts of changes in the data and metadata are automatically pushed to the Azure Data Lake without any additional configuration.

Event Based Triggers: The event based trigger in this project is created based on the official documentation provided by the Microsoft [Link](#) . In the event based trigger architecture, data integration consists of changes in the production data, consumption or reaction to certain events such as creation of new files, deletion of files or modifications of existing files. This architecture requires the triggering of pipelines that are triggered based on the events on Azure Storage accounts such as the arrival or deletion of a file in Azure Data Lake Storage Gen2.

Create a new storage event trigger

This high-level workflow describes how Data Factory interacts with Event Grid to create a storage event trigger. The data flow is the same in Azure Synapse Analytics, with Azure Synapse Analytics pipelines taking the role of the data factory in the following diagram.

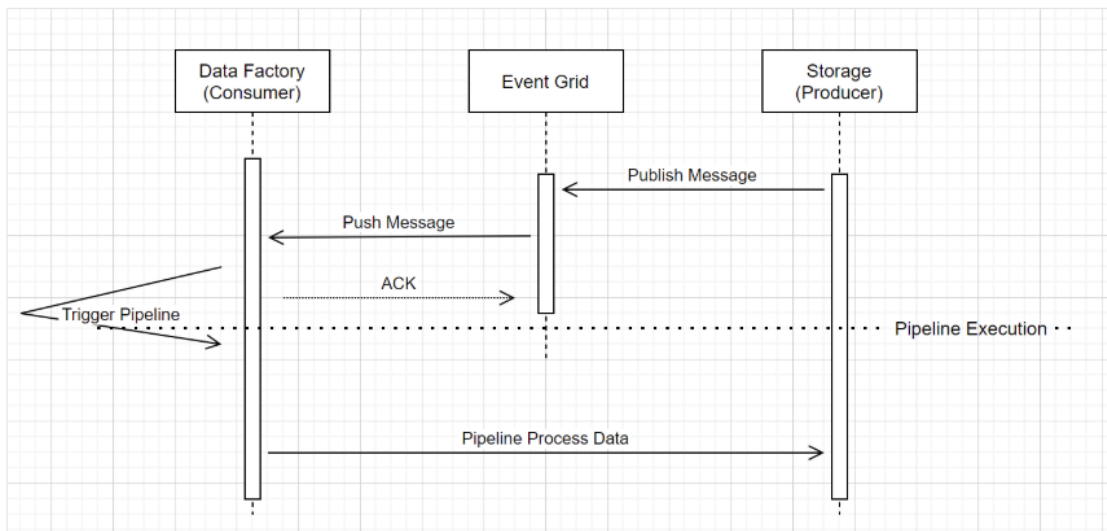


Two noticeable callouts from the workflows:

- Data Factory and Azure Synapse Analytics make *no* direct contact with the storage account. The request to create a subscription is instead relayed and processed by Event Grid. The service needs no permission to access the storage account for this step.
- Access control and permission checking happen within the service. Before the service sends a request to subscribe to a storage event, it checks the permission for the user. More specifically, it checks whether the Azure account that's signed in and attempting to create the storage event trigger has appropriate access to the relevant storage account. If the permission check fails, trigger creation also fails.

Storage event trigger pipeline run

This high-level workflow describes how storage event trigger pipelines run through Event Grid. For Azure Synapse Analytics, the data flow is the same, with Azure Synapse Analytics pipelines taking the role of Data Factory in the following diagram.



Three noticeable callouts in the workflow are related to event triggering pipelines within the service:

- Event Grid uses a Push model that relays the message as soon as possible when storage drops the message into the system. This approach is different from a messaging system, such as Kafka, where a Pull system is used.
- The event trigger serves as an active listener to the incoming message and it properly triggers the associated pipeline.
- The storage event trigger itself makes no direct contact with the storage account.
 - If you have a Copy activity or another activity inside the pipeline to process the data in the storage account, the service makes direct contact with the storage account by using the credentials stored in the linked service. Ensure that the linked service is set up appropriately.
 - If you make no reference to the storage account in the pipeline, you don't need to grant permission to the service to access the storage account.

Azure SQL Database: Event based triggers run the pipelines that consists of stored procedure queries that performs the data extraction, data manipulation and data loading into the destination tables. All the verified data is now loaded in the Azure Sql database. Azure Sql database works

as a Datawarehouse where all the historical, current and snapshot data is stored for all the reporting purposes.

Technical Implementation: Sample T-SQL Code for create Table statement and Stored Procedures based on dummy datasets.

```
CREATE TABLE dbo.FNO_Project (  
    ProjectId          NVARCHAR(20) NOT NULL,  
    ProjectName        NVARCHAR(200),  
    ProjectType        NVARCHAR(50),  
    ProjectStatus      NVARCHAR(30),  
    StartDate          DATE,  
    EndDate            DATE,  
    IsActive          BIT,  
    LastModifiedDT     DATETIME2,  
    LoadDTS           DATETIME2 DEFAULT SYSDATETIME()  
)
```

```
CREATE TABLE dbo.FNO_Account (  
    MainAccountId      NVARCHAR(20) NOT NULL,  
    AccountName        NVARCHAR(200),  
    AccountCategory    NVARCHAR(50),  
    AccountType        NVARCHAR(30),  
    IsPosting          BIT,  
    LastModifiedDT     DATETIME2,  
    LoadDTS           DATETIME2 DEFAULT SYSDATETIME(),  
    CONSTRAINT PK_FNO_Account PRIMARY KEY (MainAccountId)  
);
```

```
CREATE TABLE dbo.FNO_GLTransaction (  
    Voucher            NVARCHAR(40) NOT NULL,  
    MainAccountId      NVARCHAR(20),  
    ProjectId          NVARCHAR(20),  
    AccountingDate     DATE,  
    AmountMST          DECIMAL(18,2),  
    DebitCredit        CHAR(1),  
    LedgerAccount       NVARCHAR(100),  
    LastModifiedDT     DATETIME2,  
    LoadDTS           DATETIME2 DEFAULT SYSDATETIME(),  
    CONSTRAINT PK_FNO_GLTransaction PRIMARY KEY (Voucher, MainAccountId)  
);
```

```

CREATE OR ALTER PROCEDURE dbo.usp_Load_FNO_Project
AS
BEGIN
    SET NOCOUNT ON;

    MERGE dbo.FNO_Project AS tgt
    USING (
        SELECT
            ProjId                AS ProjectId,
            Name                   AS ProjectName,
            ProjType               AS ProjectType,
            CASE ProjStatus
                WHEN 0 THEN 'Created'
                WHEN 1 THEN 'In Progress'
                WHEN 2 THEN 'Completed'
                ELSE 'Unknown'
            END                    AS ProjectStatus,
            StartDate,
            EndDate,
            CASE WHEN ProjStatus IN (0,1) THEN 1 ELSE 0 END AS IsActive,
            ModifiedDateTime       AS LastModifiedDT
        FROM synapse_fno.ProjTable
    ) AS src
    ON tgt.ProjectId = src.ProjectId

    WHEN MATCHED THEN
        UPDATE SET
            ProjectName      = src.ProjectName,
            ProjectType      = src.ProjectType,
            ProjectStatus    = src.ProjectStatus,
            StartDate        = src.StartDate,
            EndDate          = src.EndDate,
            IsActive         = src.IsActive,
            LastModifiedDT   = src.LastModifiedDT,
            LoadDTS          = SYSDATETIME()

    WHEN NOT MATCHED THEN
        INSERT (
            ProjectId, ProjectName, ProjectType, ProjectStatus,
            StartDate, EndDate, IsActive, LastModifiedDT
        )
        VALUES (
            src.ProjectId, src.ProjectName, src.ProjectType, src.ProjectStatus,
            src.StartDate, src.EndDate, src.IsActive, src.LastModifiedDT
        );
END;

```

```
CREATE OR ALTER PROCEDURE dbo.usp_Load_FNO_Account
AS
BEGIN
    SET NOCOUNT ON;

    MERGE dbo.FNO_Account AS tgt
    USING (
        SELECT
            MainAccountId,
            Name AS AccountName,
            AccountCategory,
            AccountType,
            IsPosting,
            ModifiedDateTime AS LastModifiedDT
        FROM synapse_fno.MainAccount
    ) src
    ON tgt.MainAccountId = src.MainAccountId

    WHEN MATCHED THEN
        UPDATE SET
            AccountName = src.AccountName,
            AccountCategory = src.AccountCategory,
            AccountType = src.AccountType,
            IsPosting = src.IsPosting,
            LastModifiedDT = src.LastModifiedDT,
            LoadDTS = SYSDATETIME()

    WHEN NOT MATCHED THEN
        INSERT (
            MainAccountId, AccountName, AccountCategory,
            AccountType, IsPosting, LastModifiedDT
        )
        VALUES (
            src.MainAccountId, src.AccountName, src.AccountCategory,
            src.AccountType, src.IsPosting, src.LastModifiedDT
        );
END;
```

```

CREATE OR ALTER PROCEDURE dbo.usp_Load_FNO_GLTransaction
AS
BEGIN
    SET NOCOUNT ON;
    MERGE dbo.FNO_GLTransaction AS tgt
    USING (
        SELECT
            Voucher,
            MainAccountId,
            ProjId                AS ProjectId,
            AccountingDate,
            AmountMST,
            CASE
                WHEN AmountMST >= 0 THEN 'D'
                ELSE 'C'
            END                    AS DebitCredit,
            LedgerAccount,
            ModifiedDateTime      AS LastModifiedDT
        FROM synapse_fno.GeneralJournalAccountEntry
        WHERE AmountMST <> 0
    ) src
    ON tgt.Voucher = src.Voucher
    AND tgt.MainAccountId = src.MainAccountId
    WHEN MATCHED THEN
        UPDATE SET
            ProjectId      = src.ProjectId,
            AccountingDate = src.AccountingDate,
            AmountMST      = src.AmountMST,
            DebitCredit     = src.DebitCredit,
            LedgerAccount   = src.LedgerAccount,
            LastModifiedDT  = src.LastModifiedDT,
            LoadDTS         = SYSDATETIME()

    WHEN NOT MATCHED THEN
        INSERT (
            Voucher, MainAccountId, ProjectId,
            AccountingDate, AmountMST, DebitCredit,
            LedgerAccount, LastModifiedDT
        )
        VALUES (
            src.Voucher, src.MainAccountId, src.ProjectId,
            src.AccountingDate, src.AmountMST, src.DebitCredit,
            src.LedgerAccount, src.LastModifiedDT
        );
END;

```


Front End Applications: The front end application system uses the live data from the Azure Sql database and feeds the forms and reports created in the front end applications where the end users are able to verify and observe the changes made in the Microsoft Finance & Operations systems. The Front End Application consists of custom made reports and forms based on the business requirements that helps the Finance department and stakeholders to keep track of crucial information and key metrics.

References:

- <https://learn.microsoft.com/en-us/azure/synapse-analytics/overview-what-is>
- <https://learn.microsoft.com/en-us/power-apps/maker/data-platform/export-to-data-lake?context=%2Fazure%2Fsynapse-analytics%2Fcontext%2Fcontext>
- <https://learn.microsoft.com/en-us/azure/data-factory/how-to-create-event-trigger?tabs=data-factory>